

Python Crash Course

Agentic AI Interview Notes — concepts covered in our session

Topic	What to remember
Variables & types	Python infers types; type hints are optional.
Strings	Indexing, slicing, reversing with [::-1], f-strings.
Lists	Ordered collection; indexing, append/remove, len, loops, slicing.
Dictionaries	Key-value data; very important for JSON, APIs, tools and agent state.
Conditions & loops	if/elif/else; for/while; indentation defines blocks.
Functions	def, parameters, return values; agent tools are commonly functions.
Exceptions	try/except for handling failures safely.
Classes	__init__, self, methods — basic OOP awareness.
APIs	requests calls APIs; FastAPI is used to build APIs.
Coding patterns	Reverse, frequency map, second largest, palindrome, missing number.

1. Variables and Basic Data Types

Python does not require **let**, **const**, or an explicit variable type. The runtime determines the type from the assigned value.

```
name = "Nancy"
experience = 4
rating = 4.5
available = True
```

Common types: str = text, int = whole number, float = decimal, bool = True/False, None = no value.

```
print(type(name))  # <class 'str'>
```

Type hints

```
name: str = "Nancy"
age: int = 25

def add(a: int, b: int) -> int:
    return a + b
```

: **str** or : **int** describes the expected type. = assigns the value. -> **int** describes the expected return type.

Type hints are generally **not enforced automatically at runtime**. They improve readability, IDE assistance, static checking, and can be used by frameworks for schemas/validation.

```
def add(a: int, b: int) -> int:
    return a + b

print(add("Hello ", "Nancy"))  # works: "Hello Nancy"
```

Interview line: Python type hints do not enforce types at runtime by themselves.

2. Strings

```

text = "python"
print(text[0])      # p
print(text[-1])     # n
print(text[0:3])    # pyt
print(text[::-1])   # nohtyp

```

Slicing uses **[start:end]**: start is included and end is excluded. **[::-1]** is a common shortcut to reverse a string.

f-strings

```

name = "Nancy"
experience = 4
print(f"My name is {name} and I have {experience} years of experience")

```

3. Lists

A Python list is similar to a JavaScript array.

```

skills = ["Python", "Agentic AI", "React", "Docker"]

print(skills[0])    # Python
print(skills[-1])   # Docker
skills.append("Kubernetes")
skills.remove("React")
print(len(skills))

```

Loop through a list

```

for skill in skills:
    print(skill)

```

List slicing / reverse

```

numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])  # [20, 30, 40]
print(numbers[::-1]) # [50, 40, 30, 20, 10]

```

4. Dictionaries

A dictionary stores **key-value pairs**. Think of a JavaScript object. Dictionaries are especially important in Agentic AI because API responses, JSON-like data, tool arguments, and agent state frequently use this structure.

```

user = {
    "name": "Nancy",
    "age": 25
}

print(user["name"])
print(user.get("age"))
user["role"] = "Engineer"
user["age"] = 26

```

Agent/API-style example

```

state = {
    "user_query": "Check my balance",
    "intent": "balance_enquiry",
    "authenticated": True
}

```

5. Conditions

```

balance = 5000

if balance > 5000:
    print("High")
elif balance == 5000:
    print("Equal")

```

```
else:
    print("Low")
```

Useful operators include `==`, `!=`, `>`, `<`, `>=`, `<=`, **and**, **or**, **not**, **in**. Python uses indentation instead of braces to define code blocks.

6. Loops

```
numbers = [10, 20, 30]
for number in numbers:
    print(number)

for i in range(5):
    print(i) # 0 through 4

count = 0
while count < 3:
    print(count)
    count += 1
```

7. Functions

```
def greet(name):
    return "Hello " + name

result = greet("Nancy")
print(result)
```

def defines a function. Parameters are inputs and **return** sends a result back.

Agentic AI connection

```
def get_balance(account_id):
    # Call banking API here
    return 5000
```

A tool exposed to an agent is often implemented as a normal function. The agent/LLM decides that it needs external information, selects the tool, provides arguments, and consumes the tool result.

8. Error Handling: try / except

```
try:
    number = int("abc")
except ValueError:
    print("Invalid number")
```

Use try/except around operations that can fail, such as API calls, parsing, database access, and tool execution.

9. Classes — Basics

```
class Agent:
    def __init__(self, name):
        self.name = name

    def run(self):
        print(f"{self.name} is running")

agent = Agent("Banking Agent")
agent.run()
```

`__init__` is the initializer/constructor-style method. **self** refers to the current instance. **self.name** is an instance attribute.

10. Calling APIs with requests

requests is an HTTP client library: use it when your Python code needs to call another API.

```
import requests

response = requests.get("https://api.example.com/users")
print(response.status_code)
print(response.json())
```

POST request

```
import requests

data = {
    "name": "Nancy",
    "role": "Engineer"
}

response = requests.post(
    "https://api.example.com/users",
    json=data
)

print(response.json())
```

11. FastAPI vs requests

FastAPI	requests
Used to BUILD/expose APIs	Used to CALL/consume APIs
Web framework	HTTP client library
Server/provider side	Client/consumer side

Memory trick: FastAPI = create the API. requests = call the API.

12. Coding Practice Patterns

A. Reverse string / list

```
text = "python"
print(text[::-1]) # nohtyp

numbers = [1, 2, 3, 4]
print(numbers[::-1])
```

B. Frequency map

```
numbers = [1, 2, 2, 3, 1, 2]
freq = {}

for num in numbers:
    freq[num] = freq.get(num, 0) + 1

print(freq) # {1: 2, 2: 3, 3: 1}
```

freq.get(num, 0) returns the existing count, or 0 if the key does not exist.

C. Second largest

```
numbers = [10, 20, 50, 40, 50]
unique_numbers = list(set(numbers))
unique_numbers.sort()
print(unique_numbers[-2]) # 40
```

set() removes duplicates so a repeated maximum is not incorrectly treated as the second-largest distinct value.

D. Palindrome

```

text = "madam"

if text == text[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")

```

A palindrome reads the same forward and backward, such as **madam**, **level**, or **121**.

E. Missing number

```

numbers = [1, 2, 3, 5]
n = 5

expected_sum = n * (n + 1) // 2
actual_sum = sum(numbers)
missing = expected_sum - actual_sum
print(missing) # 4

```

13. Interview Cheat Sheet

Question	Short answer
Does Python require variable types?	No. It is dynamically typed.
What is a type hint?	An annotation describing the expected type.
Are type hints enforced at runtime?	Not by Python itself, by default.
List vs dictionary?	List = ordered values; dictionary = key-value pairs.
What does <code>[::-1]</code> do?	Returns a reversed sequence.
What does <code>range(5)</code> produce?	0, 1, 2, 3, 4.
What is <code>def</code> ?	Defines a function.
Why <code>try/except</code> ?	To handle runtime failures gracefully.
requests vs FastAPI?	requests calls APIs; FastAPI builds APIs.
Why dictionaries in agents?	Useful for JSON, tool inputs/results, state, API data.